Titulación: GRADO INGENIERÍA INFORMÁTICA Año Académico: 2024/25 -- Curso: 3º Asignatura: Competencias digitales y uso de la información	uc3m | Universidad Carlos III de Madrid

Alumno/a:	**Arturo Hernanz Carrasco-Muñoz**	**NIA:**	**100495850**



Memoria Ejercicio 1: Colas de mensajes

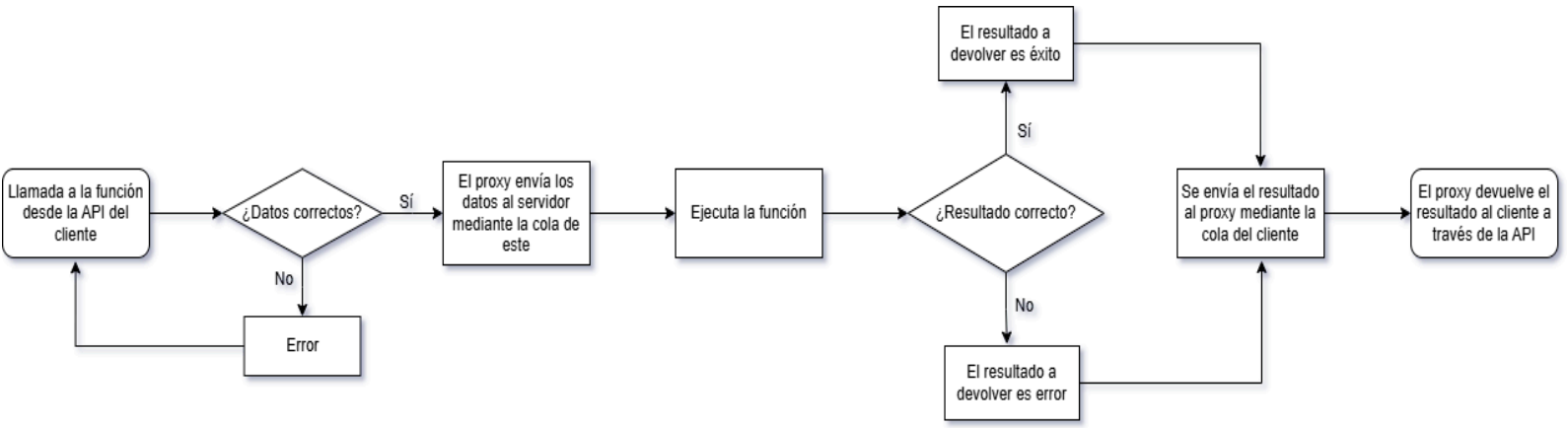
Titulación: GRADO INGENIERÍA INFORMÁTICA Año Académico: 2024/25 -- Curso: 3º Asignatura: Competencias digitales y uso de la información	uc3m | Universidad Carlos III de Madrid

Índice:

1. Decisiones de diseño destacadas
2. Compilación
3. Plan de pruebas

1. Decisiones de diseño destacadas:

El código sigue el siguiente comportamiento:



El cliente llama a las funciones de ***proxy-mq.c*** a través de la ***API***, donde a través de la cola del servidor, se le envía una estructura de tipo ***Peticion*** con los datos a ***servidor-mq.c***.
Este llama a las funciones de ***claes.c*** que devuelven 0 o -1 dependiendo de si han funcionado bien o si ha ocurrido algún error.
A continuación, el servidor envía al proxy (a través de la cola del cliente) la respuesta de estas funciones.

A continuación se detallan un poco más el comportamiento de cada componente y se explican algunas decisiones tomadas:

- **Estructura de datos seleccionada:**
  Para el desarrollo de este ejercicio se ha decidido hacer uso de listas enlazadas como estructura de datos debido a la facilidad de implementación así como la posibilidad de hacer uso de la cantidad de memoria que se necesite en cada caso.
  Aunque tienen como desventaja la complejidad O(n) para búsqueda y, en el caso de este ejercicio, para inserción, ya que, hay que asegurar que no se insertan dos tuplas para la misma clave.

- **claves.c:**
  Para el desarrollo del fichero de funciones se han implementado las funciones teniendo en cuenta que la estructura de datos es una lista enlazada.

- **proxy-mq.c:**
  Para el desarrollo del proxy del cliente se ha creado una función por cada función de ***claves.c*** que completa los campos de una estructura predefinida en el fichero ***estructuras.h*** que se le pasa a la función que se comunica con el servidor.
  He de destacar que para el caso de la función ***get_value***, como lo que tiene que recibir del servidor son los diferentes campos y no el resultado de la función (0 o -1), la cola de mensajes se crea para que estos sean estructuras de tipo ***Peticion***, una estructura que guarda los valores (con el tamaño de los objetos apuntados por los punteros al máximo para evitar violaciones de segmento) además de un campo dedicado a la operación realizada, en el resto de funciones los mensajes que llegan por la cola son enteros (0 o 1).

- **servidor-mq.c:**
  Para el desarrollo del servidor se usan hilos que manejan las peticiones de los clientes (un hilo un cliente), realizando la función deseada por el cliente y devolviendo el resultado de dicha función por la cola.
  Resalto que la función ***get_value*** se comprueba si ha fallado o no, en caso de error, se modifica el valor del ***N_value2*** cambiandolo por ***-1*** y se devuelve una estructura del tipo ***Peticion*** con los valores nulos salvo el valor mencionado previamente.
  En el resto de funciones se devuelve 0 o -1.
  En un principio se me ocurrió devolver siempre una estructura con el resultado de las funciones en un campo, sin embargo, como la mayoría de funciones solo devuelven un entero me parecía poco eficiente mandar estructuras prácticamente vacías.

2. Compilación:

Para la compilación y generación de los ejecutables del cliente y el servidor se ha hecho un makefile en el que se compilan los diferentes ficheros y se crean los ejecutables.

Se crea un ejecutable a través del fichero ***servidor-mq*** y uno por cada fichero ***app-cliente*** usados para las pruebas, para estos últimos es necesario hacer uso de la librería que llama a las funciones del fichero ***proxy-mq*** y se encargan de enviar y recibir la información del servidor.

3. Plan de pruebas:

Para probar el buen funcionamiento de este proyecto se han diseñado 7 tipos de clientes, uno por cada función a probar y uno extra para la función ***set_value***, ya que, para las pruebas es útil tener al menos dos tuplas diferentes en la base de datos.

Para probar todos estos clientes se han diseñado tres tipos de pruebas:

1. Pruebas que deben funcionar:
   - **1.1.** **set_modify_get**: Se inserta un valor en la base de datos, luego se modifica y por último se obtienen los valores de la tupla.
   - **1.2.** **set_get:** Se inserta un valor en la base de datos y luego se obtienen los valores de la tupla.
   - **1.3.** **set_get–exist:** Se inserta un valor en la base de datos y luego, de forma concurrente, se obtienen los valores de la tupla y se comprueba su existencia.
   - **1.4.** **set_get–get–get:** Se inserta un valor en la base de datos y se obtienen tres veces al mismo tiempo los valores de la tupla, verificando el buen funcionamiento de la concurrencia.
   - **1.5.** **set–set:** Se insertan dos valores diferentes en la base de datos al mismo tiempo.

2. Pruebas que deben fallar:
   - **2.1.** **destroy_get:** Se destruye la base de datos y después se intenta obtener los datos de una tupla.
   - **2.2.** **set_delete:** Se inserta un valor y después se intenta eliminar un valor que no existe en la base de datos.
   - **2.3.** **set_exist:** Se inserta un valor y después se intenta verificar la existencia de un valor que no existe en la base de datos.
   - **2.4.** **set_get:** Se inserta un valor y después se intenta obtener los valores de una tupla asociada a un valor que no existe en la base de datos.
   - **2.5.** **set_modify:** Se inserta un valor y después se intenta modificar un dato de la tupla asociada a un valor que no existe en la base de datos.

3. Pruebas que debido al uso de ***threads*** pueden o no fallar:
   - **3.1. set–delete:** Se inserta un valor a la base de datos y al mismo tiempo se borra, no se puede asegurar que la inserción se hará antes que el borrado.
   - **3.2. set–destroy:** Se inserta un valor a la base de datos y al mismo tiempo se destruye la base de datos, no se puede asegurar que la inserción se hará antes que la destrucción de la base.
   - **3.3. set–exist:** Se inserta un valor a la base de datos y al mismo tiempo se comprueba su existencia, no se puede asegurar que la inserción se hará antes que la comprobación.
   - **3.4. set–get:** Se inserta un valor a la base de datos y al mismo tiempo se obtienen los valores de la tupla, no se puede asegurar que la inserción se hará antes que la obtención.
   - **3.5. set–modify:** Se inserta un valor a la base de datos y al mismo tiempo se modifica, no se puede asegurar que la inserción se hará antes que la modificación.

Para ejecutar estas pruebas hay que seguir los siguientes pasos:

1. Ejecutar el makefile la primera vez que se realizan las pruebas:

```C/C++
make
// Exportar la libreria (Solo se hace la primera vez)
export LD_LIBRARY_PATH=.
```

2. En una terminal se ejecuta el lado del servidor:

```C/C++
./servidor
```

3. En otra terminal se ejecuta el lado del cliente:

```C/C++
// Ejecutar la prueba correspondiente
// En caso de haber algún error de permisos ejecutar:
// chmod +x <carpeta_de_pruebas>/*
./<carpeta_de_pruebas>/<fichero_a_probar>
```

En mi caso, al probar el ejercicio en ***guernika*** abrí dos terminales e inicié sesión en ambas, lo que daba error de lectura de colas, ya que, cada terminal tenía un espacio diferente.
En este caso hay que seguir estos pasos:

1. Ejecutar el makefile la primera vez que se realizan las pruebas:

```
C/C++
make
// Exportar la libreria (Solo se hace la primera vez)
export LD_LIBRARY_PATH=.
```

2. Ejecutar el servidor para que esté en segundo plano:

```
C/C++
./servidor &
```

3. Ejecutar el lado del cliente:

```
C/C++
// Ejecutar la prueba correspondiente
// En caso de haber algún error de permisos ejecutar:
// chmod +x <carpeta_de_pruebas>/*
./<carpeta_de_pruebas>/<fichero_a_probar>
```